TURNING THOUGHTS INTO CODE

The Art of Readable Code – Chapter 12

PHAN Wayotep e20220850

HEANG Denis e20220527

HUN Sopheak e20220446

NGO Seakyarith e20220185

TABLE CONTENT

1. Introduction
2. Describing Logic Clearly
3. Knowing Your Libraries Helps
4. Applying This Method to Larger Problems
5. An English Description of the Solution
6. Applying the Method Recursively

1. INTRODUCTION

The core process of writing your code:

Describe Logic

Spot Keywords

Match Code

2. DESCRIBE LOGIC CLEARLY

Example: You want to determine whether the user is authorized to see the page or not.

Logic in plain English:
There are two ways you can be authorized:
1) you are an admin
2) you own the current document (if there is one)
Otherwise, you are not authorized.

```php
if (is_admin_request()) {
    // authorized
} elseif ($document && ($document['username'] == $_SESSION['username'])) {
    // authorized
} else {
    return not_authorized();
}

// continue rendering the page ...
```

4. APPLYING THIS METHOD TO LARGER PROBLEMS

- Don't reinvent the wheel or write overly complex manual logic.
- Describe what you want to do in plain English, then look for the built-in tool.

Example (jQuery Tips Box)

Thought

"Hide the current tip, show the next one, and loop back to the start if needed."

Code

Use .next() instead of manually tracking integer IDs

jQuery

```
$('.tip').hide().eq(index).show();
index = $('.tip:visible').next().index();
if (index === -1) {
    index = 0;
}
```

3. KNOWING YOUR LIBRARIES HELPS

- Use plain English to find natural boundaries for functions.
- The main function becomes a clean, easy-to read loop.

Example (Database Rows)

English Description

Advance the rows until their times line up. Print them. Repeat until done.

Action

Extract the messy alignment logic into a helper function called AdvanceToMatchingTime()

Helper Function

```
def AdvanceToMatchingTime(a, b):
    while a.time < b.time:
        a = a.next()
    while b.time < a.time:
        b = b.next()
    return a, b
```

Main Function (Clean & Readable)

```
while not done:
    a, b = AdvanceToMatchingTime(a, b)
    print(a, b)
    a = a.next()
    b = b.next()
```

AN ENGLISH DESCRIPTION OF THE SOLUTION

Once again, let’s step back and describe in plain English what we’re trying to do:

- We are reading three row iterators in parallel.
- Whenever the rows' times don't line up, advance the rows so they do line up.
- Then print the aligned rows, and advance the rows again.
- Keep doing this until there are no more matching rows left.

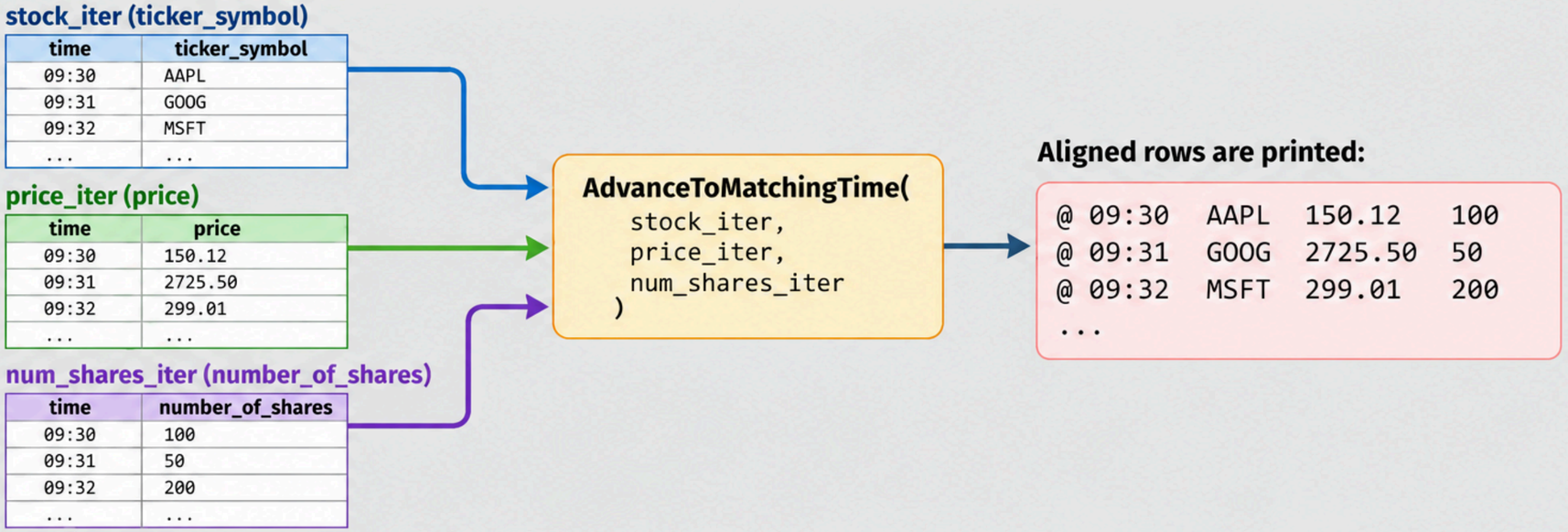
Looking back at the original code, the messiest part was the block dealing with “advance the rows so they do line up.” To present the code more clearly, we can extract all that messy logic into a new function named AdvanceToMatchingTime().

```
def PrintStockTransactions():
    stock_iter = …
    price_iter = …
    num_shares_iter = ….

    while True:
        time = AdvanceToMatchingTime(stock_iter, price_iter, num_shares_iter)
        if time is None:
            return

        # Print the aligned rows.
        print "@", time,
        print stock_iter.ticker_symbol,
        print price_iter.price,
        print num_shares_iter.number_of_shares

        stock_iter.NextRow()
        price_iter.NextRow()
        num_shares_iter.NextRow()
```



APPLYING THE METHOD RECURSIVELY

Before (Worst-Case Helper Function):

```python
def AdvanceToMatchingTime(stock_iter, price_iter, num_shares_iter):
    while stock_iter and price_iter and num_shares_iter:
        stock_time = stock_iter.time
        price_time = price_iter.time
        num_shares_time = num_shares_iter.time

        # Skip over the oldest row
        if stock_time != price_time or stock_time != num_shares_time:
            if stock_time <= price_time and stock_time <= num_shares_time:
                stock_iter.NextRow()
            elif price_time <= stock_time and price_time <= num_shares_time:
                price_iter.NextRow()
            elif num_shares_time <= stock_time and num_shares_time <= price_time:
                num_shares_iter.NextRow()
        continue
    return stock_time
```

Issues:

- Tricky, repetitive <= comparisons.
- Cluttered with domain-specific names (stock_iter, price_iter).

After (Clean & General Code):

```python
def AdvanceToMatchingTime(row_iter1, row_iter2, row_iter3):
    while row_iter1 and row_iter2 and row_iter3:
        t1, t2, t3 = row_iter1.time, row_iter2.time, row_iter3.time
        if t1 == t2 == t3:
            return t1

        tmax = max(t1, t2, t3)

        # If any row is "behind," advance it.
        if t1 < tmax: row_iter1.NextRow()
        if t2 < tmax: row_iter2.NextRow()
        if t3 < tmax: row_iter3.NextRow()
    return None
```

Key Improvements:

- Uses max() instead of complex if/elif chains.
- Uses short, generic names (row_iter1, t1) decoupled from database columns.

THANK YOU